

Strings in Java

Character:

↳ 'A', 'a' ... 1, 2, 3, ... 0 ... '\'

String: Sequence of characters

↳ `char[]` → ['a', 'd', 'i', 't', 'y', 'a']

↳ comparison (X)

↳ concat (X)

↳ substring (X)

} no direct way → we've to write manually in char array

`int` → Integer

`char` → Character

`Float` → Float

`char[]` ^{analogy} → String

String → `java.lang.String`

↳ auto imported package

```
class String {  
    //  
  
}
```

`String s = new String();`

`String s = "Hello";`

`Integer i = 4;`

↳ `new Integer(4);`

↳ `Integer.valueOf(4);`

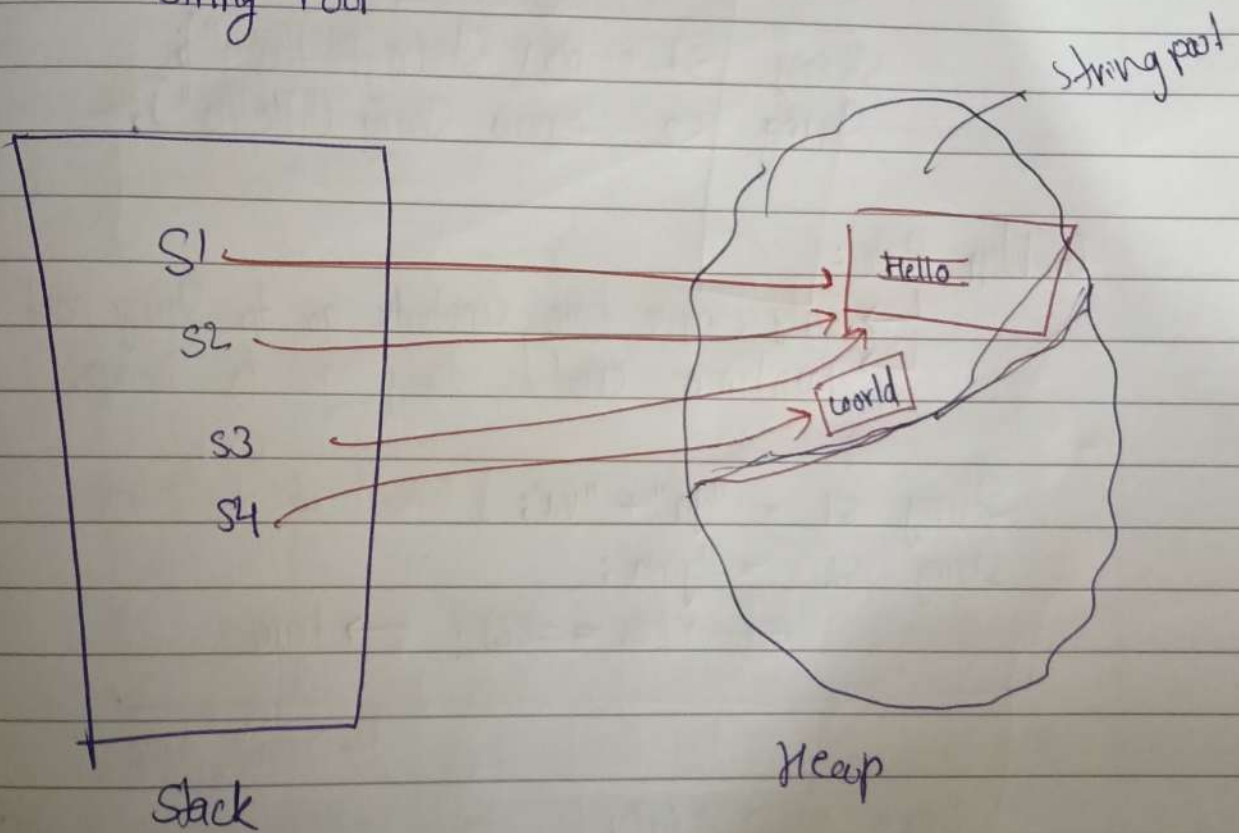
String

- Immutable class
- passwords / URL's / Hashes

String declaration

Literal~~new~~ new

String Pool



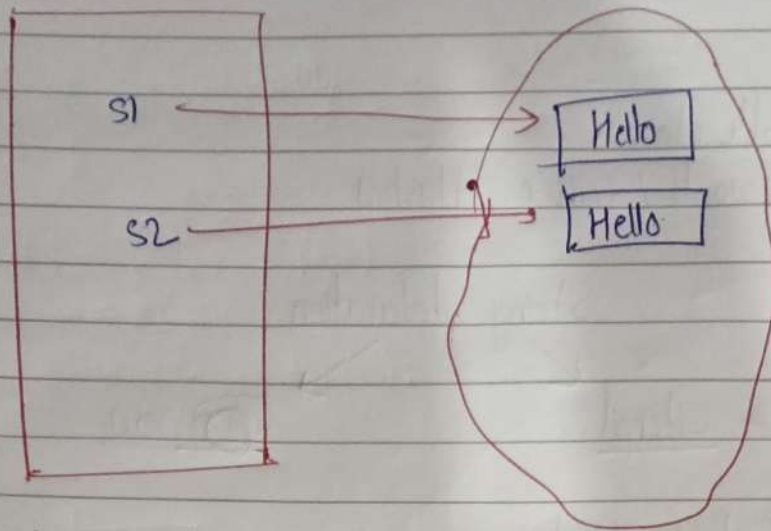
```
String s1 = "Hello";
```

→ Literal

```
String s2 = "Hello";
```

```
String s3 = "Hello";
```

```
String s4 = "world";
```

```
String s1 = new String("Hello");
String s2 = new String("Hello");
```

Golden Rule:

- only compile time constants go to String pool automatically.
- Runtime created string go to heap.

```
String s1 = "ja" + "va"; } Concat at compile time
```

```
String s2 = "java";
```

→ (s1 == s2)? → true

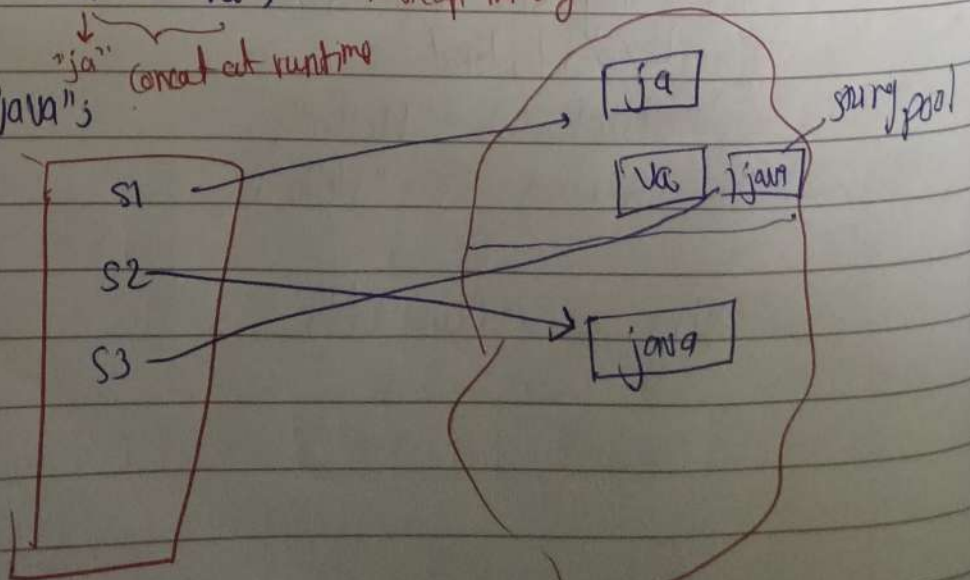
Compile time

```
String s1 = "ja";
```

```
String s2 = s1 + "va"; // Heap memory
```

```
String s3 = "java";
```

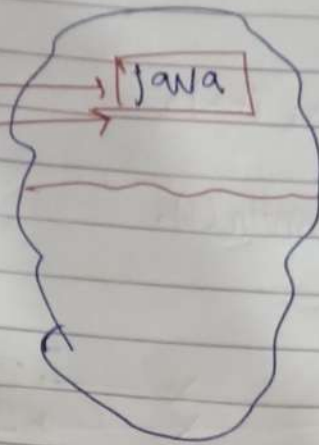
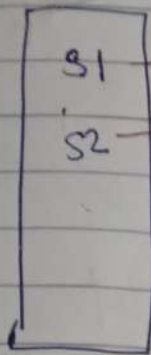
s2 == s3 // false



String s1 = "java's"
String s2 = s1;

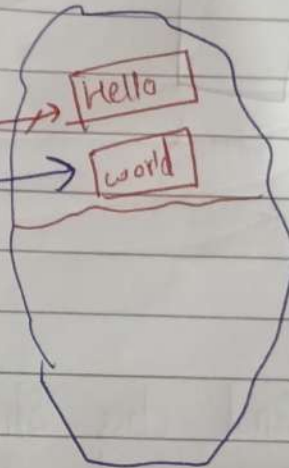
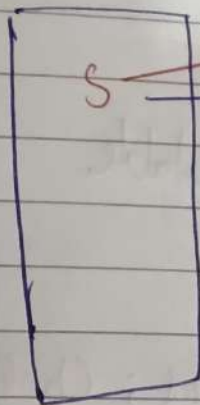
compile time

(s1 == s2) // true

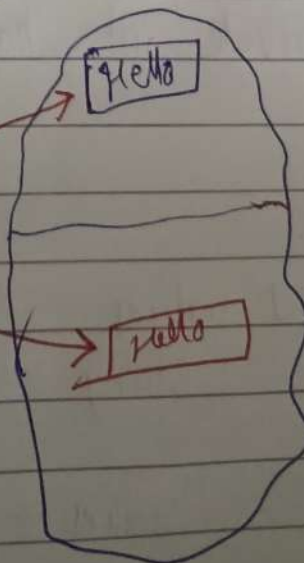
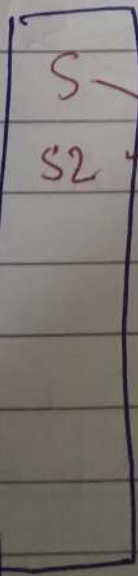


string pool

String s = "Hello";
s = "world";



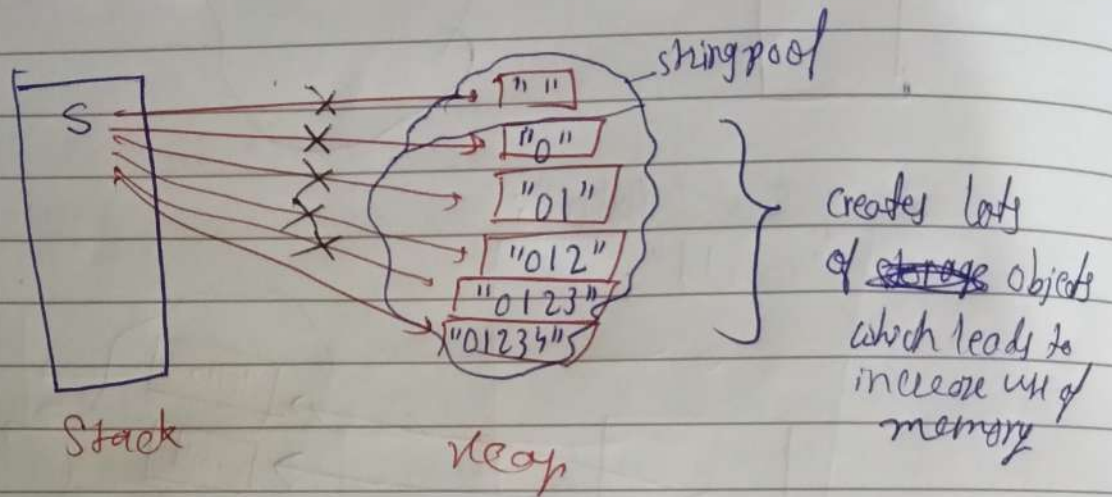
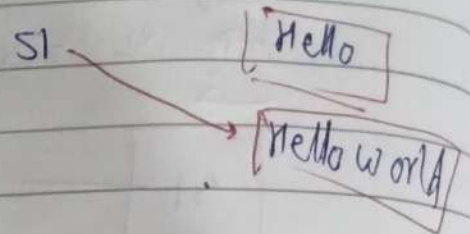
String s = new String("Hello");



String s2 = "Hello";

Problem of Immutability

```
String s = "";
for (int i=0; i<5; i++) {
    s += "i";
    System.out.println(s);
}
```

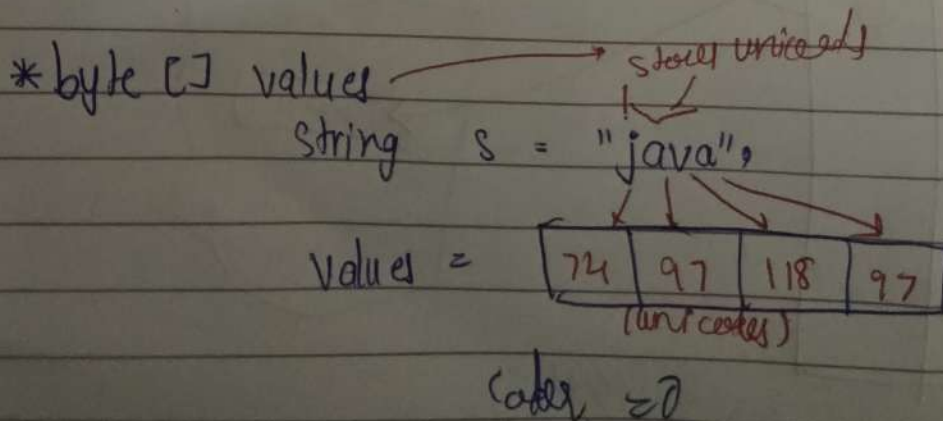


* String Class

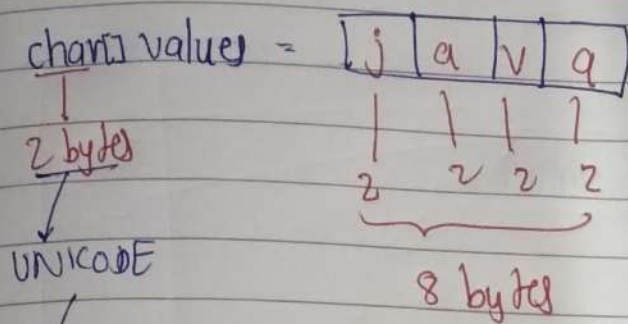
```
public final class String {
    x private final char[] values; (before Java 9)
    ✓ private final byte[] values; (after Java 9)
    private final byte coder;
    private int hash;
}
```

Immutable

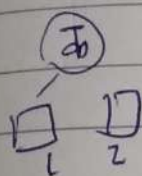
`s = "java";`
`s.hashCode()` → int



* Before Java 9
String s = "java";

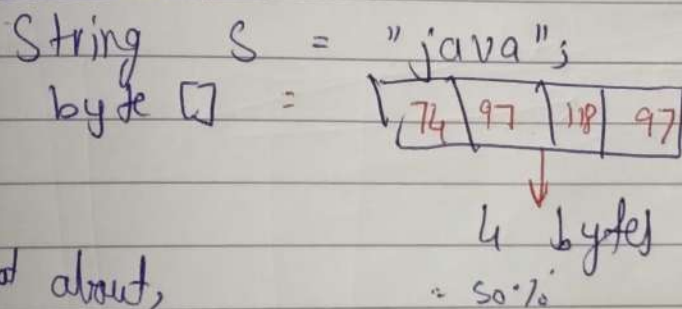


(a=2, A=3, 0=...) but they all could be represented in ASCII values which takes 1 byte



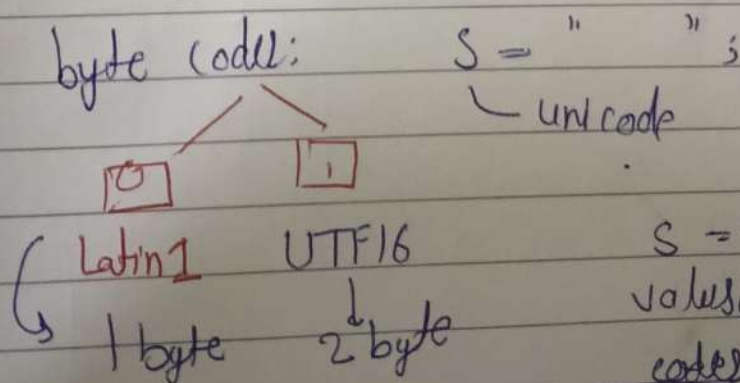
most of the time we use ASCII not unicode. so why not take byte array it generally takes less memory than char

from Java 9



but what about,

s = "कट"



s = "java"
values[] = [74, 97, 118, 97]
codes = 0

s = "कट"
values[] = [0, 1]
codes = 1

String optimization (Immutable)

- String Pool
- char [] → byte []
- Caching the hash value